AI-Powered HRMS

Face Recognition, RAG & Agentic AI

Project Documentation — Architecture, Workflow & Setup Guide

Personal Project | Sriram G

1. Project Overview

This is a full-stack Human Resource Management System (HRMS) that combines computer vision, a role-based web application, and three GenAI features. It was built to bridge existing computer vision engineering experience with production-style Generative AI patterns — Retrieval-Augmented Generation (RAG), agentic tool-calling, and LLM cost optimization.

Motivation: most current job descriptions in the ML/AI space now expect familiarity with LLMs, RAG, and AI agents, in addition to traditional ML/CV skills. Rather than starting a new project from scratch, an existing, working face-recognition attendance system was extended with a GenAI layer — combining both skill sets in a single, coherent product.

Who does what

Role	Capabilities
Admin	Manage employees, monitor attendance, approve leave, generate salary slips, upload HR policy documents, ask the AI agent natural-language questions about workforce data, and track GenAI usage cost.
Employee	Check in/out (face recognition or manual), apply for leave, view salary slips, ask the HR policy chatbot questions, and manage their profile.

2. System Architecture

The system is organized into four layers:

1. Client layer — a React (Vite + Tailwind) single-page app with separate Admin and Employee dashboards, plus a standalone Python script that performs face recognition on a webcam feed and calls the backend over HTTP.

2. API layer — a FastAPI backend exposing role-based REST endpoints, organized into three logical modules: Auth & CRUD, RAG engine, and AI agent.

3. Data layer — MySQL for all structured records (employees, attendance, leave, salary, chat logs), and ChromaDB for HR policy document embeddings.

4. External services — the Groq API for LLM inference, and a Redis/Valkey cache to avoid repeat LLM calls for repeated questions.

Component summary

Component	Responsibility
Auth & CRUD	JWT-based login, role guards (admin/employee), employee/attendance/leave/salary management
RAG engine	PDF chunking, embedding generation, ChromaDB storage and retrieval, cache-checked Groq calls for HR policy Q&A
AI agent	Interprets admin questions, selects one of four fixed database-query tools, and formats a natural-language answer
MySQL	System of record for all employee, attendance, leave, salary, and chat log data
ChromaDB	Persistent local vector store for HR policy document chunks

Valkey (Redis)	Exact-match cache for RAG answers — repeated questions cost $0 on a cache hit
Groq API	Hosted LLM inference (model: openai/gpt-oss-120b)

3. Feature Workflows

3.1 HR Policy Chat (RAG)

1. Admin uploads an HR policy PDF once, via the HR Policy Management screen.

2. The backend extracts text, splits it into overlapping chunks, generates embeddings (sentence-transformers, local and free), and stores them permanently in ChromaDB.

3. An employee asks a question in the HR Policy Chat widget.

4. The backend first checks the Valkey cache for an exact match on that question. On a cache hit, the stored answer is returned immediately at zero additional cost.

5. On a cache miss, the backend retrieves the most relevant policy chunks from ChromaDB, sends them to Groq as grounding context, and asks for an answer.

6. The answer, token usage, and cost are logged to the chat_logs table and also written to the cache (1-hour TTL) before being returned to the employee.

3.2 AI Admin Agent

1. An admin asks a natural-language question, e.g. "Who was absent today?"

2. The request is sent to Groq along with four tool definitions: get_attendance_summary, get_employee_attendance, get_pending_leaves, and get_salary_info.

3. Groq decides which tool applies and with what arguments — it does not write raw SQL. This is a deliberate safety choice: a tool-calling loop against pre-written, parameterized functions cannot generate an unsafe or incorrect query the way free-form LLM-generated SQL could.

4. The backend executes the selected Python function directly against MySQL and returns the structured result to Groq.

5. Groq turns that structured result into a plain-English answer, which is logged (chat_type = admin_agent) and returned to the admin.

3.3 Face Recognition Attendance

1. A standalone Python script (OpenCV DNN face detector + Dlib face recognition model) runs against a webcam feed.

2. On recognizing a known face, it calls the backend's /attendance/face-checkin or /attendance/face-checkout endpoint, authenticated with a device-level API key rather than an employee JWT — this mirrors how a physical kiosk device would authenticate itself.

3. The backend applies the same business rules as manual check-in: duplicate check-in prevention, and a 10:00 AM check-in cutoff enforced both in the UI and on the server.

4. A daily background job (APScheduler) automatically marks any employee with no attendance record as absent at 10:00 AM, independent of whether the face-recognition client is running.

4. Technology Stack

Layer	Technology
Frontend	React (Vite), Tailwind CSS, Recharts, Axios
Backend	FastAPI, SQLAlchemy, Pydantic, APScheduler

Database	MySQL
Face Recognition	Dlib, OpenCV (DNN face detector)
Vector Store	ChromaDB (persistent, local)
Embeddings	sentence-transformers (local, no API cost)
LLM	Groq API — model: openai/gpt-oss-120b
Cache	Redis / Valkey
Auth	JWT (python-jose), bcrypt password hashing

5. Local Setup (Without Docker)

This project runs entirely on locally installed services — no containerization is required for local development.

5.1 Prerequisites

- Python 3.11 (via Conda or venv)
- Node.js 20+
- MySQL Server (running locally)
- Valkey or Redis (running locally, e.g. via a native Windows build or WSL)
- A free Groq API key (console.groq.com)

5.2 Backend Setup

```
cd attendance-backend
conda create -n attendance-system python=3.11
conda activate attendance-system
pip install -r requirements.txt
cp .env.example .env   # fill in DATABASE_URL, SECRET_KEY, GROQ_API_KEY, DEVICE_API_KEY
mysql -u root -p < schema.sql
uvicorn app.main:app --reload
```

Backend API and interactive docs available at http://localhost:8000/docs

5.3 Frontend Setup

```
cd attendance-frontend
npm install
npm run dev
```

Frontend available at http://localhost:5173

5.4 Face Recognition Client

```
cd face-recognition-client
python main_attendance.py
```

Requires the Dlib shape predictor and face recognition model files, plus a features_all.csv of known face encodings labeled Name_EmployeeID.

5.5 Environment Variables

```
DATABASE_URL=mysql+pymysql://root:<password>@localhost:3306/attendance_system
SECRET_KEY=<random string>
ALGORITHM=HS256
```

```
ACCESS_TOKEN_EXPIRE_MINUTES=480
GROQ_API_KEY=<your groq api key>
DEVICE_API_KEY=<random string for the face-recognition client>
```

6. Key Design Decisions

Why tool-calling instead of LLM-generated SQL

Letting an LLM write and execute raw SQL creates real risk — incorrect joins, unintended data exposure, or destructive statements. Restricting the agent to four fixed, parameterized Python functions keeps all database logic under direct control, while the LLM's role is limited to reasoning about which tool fits the question.

Why cache the RAG chatbot but not the AI agent

HR policy answers are stable and frequently repeated across employees, making caching high-value. Admin agent questions are often date-sensitive (e.g. "who was absent today") and would risk returning stale answers if cached with a long TTL, so the agent currently calls Groq fresh on every request.

Why a device API key for face-recognition check-in

The face-recognition client identifies employees itself, before any login occurs, so it cannot present an employee's personal JWT. A separate device-level API key authenticates the client as a trusted kiosk endpoint, distinct from user authentication.

Model note

The project originally used Groq's llama-3.3-70b-versatile model. That model was deprecated by Groq (shutdown announced for August 16, 2026), and the project was migrated to openai/gpt-oss-120b, which is also cheaper per token and built with native tool-calling support.